

Prompt: 4 Fixes — Canal Agencia + Confirmación + Drag&Drop Fecha

Fix 1 — Agregar “Agencia” como opción de canal al crear reserva

Problema

Al crear una reserva, el campo “Canal” no tiene la opción “Agencia”. Hay que agregarla en los dos formularios donde aparece el selector de canal.

1A — En `QuickReservationDialog` (planning)

Buscar el array `sourceOptions`:

```
const sourceOptions = [
  { value: "directo", label: "Directo" },
  { value: "telefono", label: "Teléfono" },
  { value: "web", label: "Web" },
  { value: "booking", label: "Booking" },
  { value: "expedia", label: "Expedia" },
  { value: "airbnb", label: "Airbnb" },
  { value: "despegar", label: "Despegar" },
  { value: "empresa", label: "Empresa" },
];
```

Reemplazar por:

```
const sourceOptions = [
  { value: "directo", label: "Directo" },
  { value: "telefono", label: "Teléfono" },
  { value: "web", label: "Web" },
  { value: "booking", label: "Booking" },
  { value: "expedia", label: "Expedia" },
  { value: "airbnb", label: "Airbnb" },
  { value: "despegar", label: "Despegar" },
  { value: "empresa", label: "Empresa" },
  { value: "agencia", label: "Agencia de Viajes" }, // ← agregar
```

```
];
```

1B — En `ReservationFormDialog` (página de Reservas)

Buscar el `<SelectContent>` del campo `Canal/Source` que tiene estas opciones:

```
<SelectedItem value="directo">Directo</SelectedItem>
<SelectedItem value="web">Web</SelectedItem>
<SelectedItem value="ota">OTA (Booking, etc.)</SelectedItem>
<SelectedItem value="empresa">Empresa</SelectedItem>
<SelectedItem value="telefono">Teléfono</SelectedItem>
```

Reemplazar por:

```
<SelectedItem value="directo">Directo</SelectedItem>
<SelectedItem value="telefono">Teléfono</SelectedItem>
<SelectedItem value="web">Web</SelectedItem>
<SelectedItem value="booking">Booking</SelectedItem>
<SelectedItem value="expedia">Expedia</SelectedItem>
<SelectedItem value="airbnb">Airbnb</SelectedItem>
<SelectedItem value="despegar">Despegar</SelectedItem>
<SelectedItem value="ota">OTA (otros)</SelectedItem>
<SelectedItem value="empresa">Empresa</SelectedItem>
<SelectedItem value="agencia">Agencia de Viajes</SelectedItem>    {/* ← agregar */}
```

1C — En `shared/schema.ts`, agregar "agencia" al tipo `ReservationSource`

Buscar:

```
export type ReservationSource = "directo" | "web" | "booking" | "expedia" | "airb
```

Reemplazar por:

```
export type ReservationSource = "directo" | "web" | "booking" | "expedia" | "airb
```

Fix 2 — Confirmación PDF: mostrar early check-in y late check-out con costos

Problema

El PDF de confirmación muestra la tarifa de habitación y el total, pero no menciona early check-in ni late check-out aunque estén activos en la reserva, ni sus costos.

Cambio en `printConfirmation()` dentro de `ReservationDetailModal`

Buscar donde se declaran las variables antes del HTML:

```
const company = (reservation as any).company?.razonSocial || ...
```

Agregar después de esa línea:

```
const earlyCheckIn = reservation.earlyCheckIn;
const earlyCheckInTime = reservation.earlyCheckInTime || "";
const earlyCheckInCharge = reservation.earlyCheckInCharge
  ? parseFloat(String(reservation.earlyCheckInCharge))
  : 0;

const lateCheckOut = reservation.lateCheckOut;
const lateCheckOutTime = reservation.lateCheckOutTime || "";
const lateCheckOutCharge = reservation.lateCheckOutCharge
  ? parseFloat(String(reservation.lateCheckOutCharge))
  : 0;

const formatMoney = (n: number) =>
  `$$${n.toLocaleString("es-AR", { minimumFractionDigits: 2 })}`;

const totalConExtras =
  parseFloat(reservation.totalRoomAmount || "0") +
  (earlyCheckIn ? earlyCheckInCharge : 0) +
  (lateCheckOut ? lateCheckOutCharge : 0);

const totalConExtrasStr = formatMoney(totalConExtras);
```

En el HTML del PDF, buscar el bloque de la `rates-grid` que contiene la tarifa diaria y el total:

```
<div class="rates-grid">
  <div class="rate-cell rate-label">Tarifa diaria / Daily rate</div><div class="r
  <div class="rate-cell rate-label">Tarifa diaria Cochera / Garage rate</div><div
  <div class="rate-cell rate-total">Total Alojamiento / Total rate</div><div clas
  <div class="rate-cell rate-label">Total Cochera / Garage</div><div class="rate-
</div>
<div class="total-box"><span>Tarifa Total / Total rate:</span>${totalRate}</div>
```

Reemplazar por (usando template literals de JS, con backticks):

```
`<div class="rates-grid">
  <div class="rate-cell rate-label">Tarifa diaria / Daily rate</div>
  <div class="rate-cell rate-value">${dailyRate}</div>
  <div class="rate-cell rate-label">Total Alojamiento / Total room rate</div>
  <div class="rate-cell rate-value">${totalRate}</div>
  ${earlyCheckIn ? `
    <div class="rate-cell rate-label" style="color:#b45309;">Early Check-in${earlyC
  <div class="rate-cell rate-value" style="color:#b45309;">${formatMoney(earlyChe
    ` : ""}
  ${lateCheckOut ? `
    <div class="rate-cell rate-label" style="color:#7c3aed;">Late Check-out${lateCh
  <div class="rate-cell rate-value" style="color:#7c3aed;">${formatMoney(lateChec
    ` : ""}
</div>
<div class="total-box">
  <span>Tarifa Total / Total rate:</span>${totalConExtrasStr}
</div>`
```

También actualizar la condición en `conditions` (el texto de las condiciones) para que diga el horario correcto si hay early/late:

```
// Buscar esta línea en conditions:
`<p>Nuestro horario de Check in es a partir de las 15:00 Hs y el Check out es has

// Reemplazar por:
`<p>Nuestro horario de Check in es a partir de las ${earlyCheckIn && earlyCheckIn
```

Fix 3 y 4 — Drag & drop: mover a otra habitación Y otra fecha

Problema actual

- El drop target es la fila (`DroppableRoomRow`), que solo sabe el `roomId` . No sabe a qué columna de fecha se soltó.
- Por eso `handleDragEnd` detecta `roomId` de destino pero nunca detecta fecha de destino.
- La mutación solo manda `{ roomId }` al PATCH, nunca manda las nuevas fechas.

- Resultado: al arrastrar a otra columna de fecha, cambia la habitación pero no las fechas.

Solución: cambiar drop targets de filas a celdas individuales

Paso A — Eliminar `DroppableRoomRow` como zona de drop y hacerlo solo visual

Cambiar `DroppableRoomRow` para que ya no use `useDroppable`, solo sea un `<tr>` normal con la clase de highlight:

```
function DroppableRoomRow({
  roomId,
  children,
  className,
  isOver,
  "data-testid": testId,
}): {
  roomId: string;
  children: React.ReactNode;
  className?: string;
  isOver?: boolean;
  "data-testid"?: string;
}) {
  // YA NO usa useDroppable — solo es un wrapper visual
  return (
    <tr
      className={`${className} || ""} ${isOver ? "bg-primary/10 ring-1 ring-primar
      data-testid={testId}
    >
      {children}
    </tr>
  );
}
```

Paso B — Crear componente `DroppableCell` para cada celda de la grilla

Agregar este nuevo componente junto a los otros:

```
function DroppableCell({
  roomId,
  day,
  children,
  className,
}): {
  roomId: string;
  day: string;
```

```

    children: React.ReactNode;
    className?: string;
  }) {
    const { setNodeRef, isOver } = useDroppable({
      id: `drop-${roomId}-${day}`,
      data: { roomId, day },
    });

    return (
      <td
        ref={setNodeRef}
        className={` ${className} || ""} ${isOver ? "bg-primary/10 ring-2 ring-primar
      >
        {children}
      </td>
    );
  }

```

Paso C — Reemplazar `<td>` de cada celda de la grilla por `<DroppableCell>`

En el render de la grilla, buscar el `<td>` que envuelve cada celda de día (el que tiene `key={day}` **y** `className="p-0.5 border-b ..."`):

```

// ANTES — td normal
<td
  key={day}
  className={`p-0.5 border-b ${info.isToday ? "bg-primary/5" : ""}`}
>
  {/* ... contenido ... */}
</td>

// DESPUÉS — DroppableCell
<DroppableCell
  key={day}
  roomId={room.id}
  day={day}
  className={`p-0.5 border-b ${info.isToday ? "bg-primary/5" : ""}`}
>
  {/* ... mismo contenido ... */}
</DroppableCell>

```

Paso D — Actualizar `handleDragEnd` para leer `roomId` Y `day` del drop target

Reemplazar el `handleDragEnd` completo:

```

const handleDragEnd = (event: DragEndEvent) => {

```

```

setDragActiveId(null);
const { active, over } = event;
if (!over || !data) return;

const reservationId = (active.data.current as any)?.reservationId;
const fromRoomId = (active.data.current as any)?.roomId;
const toRoomId = (over.data.current as any)?.roomId;
const toDay = (over.data.current as any)?.day as string | undefined; // ← nuev

if (!reservationId || !toRoomId) return;

const reservation = data.reservations[reservationId];
if (!reservation) return;

const toRoom = data.rooms.find(r => r.id === toRoomId);
const fromRoom = data.rooms.find(r => r.id === fromRoomId);
if (!toRoom || !fromRoom) return;

// Calcular nuevas fechas si se arrastró a otra columna
let newCheckIn = reservation.checkIn;
let newCheckOut = reservation.checkOut;

if (toDay && toDay !== reservation.checkIn) {
  // El usuario soltó en una celda diferente → calcular offset de días
  const originalCheckIn = new Date(reservation.checkIn + "T12:00:00");
  const originalCheckOut = new Date(reservation.checkOut + "T12:00:00");
  const nights = Math.round(
    (originalCheckOut.getTime() - originalCheckIn.getTime()) / (1000 * 60 * 60)
  );
  newCheckIn = toDay;
  const newCheckOutDate = new Date(toDay + "T12:00:00");
  newCheckOutDate.setDate(newCheckOutDate.getDate() + nights);
  newCheckOut = newCheckOutDate.toISOString().split("T")[0];
}

// Verificar conflictos en la habitación destino con las nuevas fechas
for (const [cellDay, existingResId] of Object.entries(data.cellReservations[toR
  if (existingResId === reservationId) continue;
  if (cellDay >= newCheckIn && cellDay < newCheckOut) {
    toast({
      title: "Habitación ocupada",
      description: `La habitación ${toRoom.roomNumber} tiene otra reserva en es
      variant: "destructive",
    });
    return;
  }
}

```

```

// Si es la misma habitación Y la misma fecha → no hacer nada
if (fromRoomId === toRoomId && newCheckIn === reservation.checkIn) return;

setMoveConfirm({
  reservationId,
  guestName: reservation.guestName,
  fromRoomNumber: fromRoom.roomNumber,
  toRoomId: toRoom.id,
  toRoomNumber: toRoom.roomNumber,
  toRoomType: toRoom.roomType.name,
  newCheckIn,      // ← nuevo
  newCheckOut,     // ← nuevo
  dateChanged: newCheckIn !== reservation.checkIn, // ← nuevo
});
};

```

Paso E — Actualizar el tipo de `moveConfirm` para incluir fechas

```

const [moveConfirm, setMoveConfirm] = useState<{
  reservationId: string;
  guestName: string;
  fromRoomNumber: string;
  toRoomId: string;
  toRoomNumber: string;
  toRoomType: string;
  newCheckIn: string;      // ← agregar
  newCheckOut: string;     // ← agregar
  dateChanged: boolean;    // ← agregar
} | null>(null);

```

Paso F — Actualizar `moveReservationMutation` para enviar fechas

```

const moveReservationMutation = useMutation({
  mutationFn: async ({
    reservationId,
    roomId,
    checkInDate,
    checkOutDate,
  }: {
    reservationId: string;
    roomId: string;
    checkInDate?: string;
    checkOutDate?: string;
  }) => {

```



```

const payload: Record<string, string> = { roomId };
if (checkInDate) payload.checkInDate = checkInDate;
if (checkOutDate) {
  payload.checkOutDate = checkOutDate;
  // Recalcular nights
  const ci = new Date(checkInDate! + "T12:00:00");
  const co = new Date(checkOutDate + "T12:00:00");
  payload.nights = String(Math.round((co.getTime() - ci.getTime()) / (1000 *
}
const res = await apiRequest("PATCH", `/api/reservations/${reservationId}`, p
return res.json());
},
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ["/api/planning"] });
  queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
  toast({ title: "Reserva movida", description: "La habitación y fechas fueron
  setMoveConfirm(null);
},
onError: (error: any) => {
  const msg = error?.message || "No se pudo mover la reserva.";
  toast({ title: "Error", description: msg, variant: "destructive" });
},
});

```

Paso G — Actualizar el diálogo de confirmación para mostrar también el cambio de fecha

En el `Dialog` de confirmación de movimiento, agregar info de fechas si cambiaron:

```

{moveConfirm && (
  <div className="space-y-3 py-2">
    <div className="flex items-center gap-2">
      <User className="h-4 w-4 text-muted-foreground" />
      <span className="font-medium">{moveConfirm.guestName}</span>
    </div>

    {/* Cambio de habitación */}
    <div className="flex items-center gap-3">
      <Badge variant="outline" className="text-sm">Hab. {moveConfirm.fromRoomNumb
      <ArrowLeftRight className="h-4 w-4 text-muted-foreground" />
      <Badge className="text-sm bg-primary">Hab. {moveConfirm.toRoomNumber}</Badg
    </div>

    {/* Cambio de fecha — solo si cambió */}
    {moveConfirm.dateChanged && (
      <div className="flex items-center gap-3 text-sm">

```

```

        <Calendar className="h-4 w-4 text-muted-foreground" />
        <span className="text-muted-foreground">Nuevas fechas:</span>
        <Badge variant="outline" className="text-orange-600 border-orange-300">
            {moveConfirm.newCheckIn} → {moveConfirm.newCheckOut}
        </Badge>
    </div>
  )}

  <p className="text-sm text-muted-foreground">Tipo: {moveConfirm.toRoomType}</p>
</div>
)}

```

Y el botón de confirmar debe pasar las nuevas fechas:

```

<Button
  onClick={() => {
    if (moveConfirm) {
      moveReservationMutation.mutate({
        reservationId: moveConfirm.reservationId,
        roomId: moveConfirm.toRoomId,
        checkInDate: moveConfirm.newCheckIn,    // ← pasar siempre
        checkOutDate: moveConfirm.newCheckOut,  // ← pasar siempre
      });
    }
  }}
  disabled={moveReservationMutation.isPending}
  data-testid="button-confirm-move"
>
  {moveReservationMutation.isPending ? "Moviendo..." : "Confirmar Movimiento"}
</Button>

```

Resumen de archivos modificados

Cambio	Archivo
Agregar "agencia" en sourceOptions	planning.tsx (QuickReservationDialog)
Agregar "agencia" en SelectContent	reservations.tsx (ReservationFormDialog)
Agregar "agencia" al tipo	shared/schema.ts
Early/late en PDF confirmación	planning.tsx (printConfirmation)

DroppableCell por columna de fecha	<code>planning.tsx</code>
handleDragEnd calcula nuevas fechas	<code>planning.tsx</code>
moveConfirm incluye newCheckIn/Out	<code>planning.tsx</code>
moveReservationMutation envía fechas	<code>planning.tsx</code>
Diálogo muestra cambio de fecha	<code>planning.tsx</code>